

Reviewer Pack — Minimal Rank of Quaternionic Representations in Boolean Alge...

Entry 95d8ba0e68c2 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-26 18:54:21 UTC

Minimal Rank of Quaternionic Representations in Boolean Algebras vs XOR-AND Tree Width

Entry ID: 95d8ba0e68c2 **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-26 18:54:21 UTC

1. Conjecture

Field A (mathematical branch): Representation Theory (Quaternionic Representations) **Field B** (complexity object): Complexity Theory: XOR-AND Tree Width

Statement:

[‘For a Boolean function f with n variables, the minimal rank of its quaternionic representation over the complex numbers is upper-bounded by the XOR-AND tree width of f .’, ‘Equivalently, for any QuaternionicRepresentation(Q) of f , there exists an XOR-and Tree(T) such that Q is isomorphic to a sub-representation of T ’s associated boolean algebra.”]

Rationale (proposer’s reasoning):

[‘Quaternionic representations provide a different mathematical framework that could potentially expose deeper structures in Boolean functions, which are not captured by traditional representation theories.’, ‘The use of XOR-AND trees as a complexity measure for boolean functions is well-established and provides a concrete connection point with computational complexity theory.’]

Taxonomy category: QuaternionicRepresentation (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): c5fd7f10a605a055

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if, for all Boolean functions f with n variables (where n ranges from 10 to 20), the minimal rank of its quaternionic representation Q is less than or equal to the XOR-AND tree width of the boolean algebra generated by f .

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	UNCERTAIN	1.00	UNCERTAIN	HITS
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	UNCERTAIN	1.00	HITS	UNCERTAIN
KARP_LIPTON	SAFE	0.50	UNCERTAIN	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 7 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - "minimal rank" quaternionic representation "XOR-AND tree width"
- "complex numbers" minimal rank quaternionic representation Boolean algebra XOR-AND -
"isomorphic sub-representation" quaternionic representation XOR-and tree boolean algebra

Top relevant hits considered: - [<http://arxiv.org/abs/1101.2456v3>] Polynomial representations and categorifications of Fock Space - [<http://arxiv.org/abs/2412.13827v1>] Bounds for the Zeros of Quaternionic Polynomials and Regular Functions Using Matrix Techniques - [<http://arxiv.org/abs/0711.0795v4>] Finite-Dimensional Representations of Hyper Loop Algebras Over Non-Algebraically Closed Fields - [<http://arxiv.org/abs/1005.0140v4>] Representations of hom-Lie algebras - [<http://arxiv.org/abs/math/0405176v4>] Quantized symplectic oscillator algebras of rank one - [<http://arxiv.org/abs/1102.1242v2>] A refinement of Stone duality to skew Boolean algebras - [<http://arxiv.org/abs/0807.4776v2>] Center and representations of infinitesimal Hecke algebras of sl_2

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.2s

5.1 Generated Python source

```
import random
import math

def gcd(a, b):
    while b:
        a, b = b, a % b
    return a

def lcm(a, b):
    return abs(a * b) // gcd(a, b)

def matrix_multiply(A, B):
    m, n, p = len(A), len(B), len(B[0])
    C = [[0] * p for _ in range(m)]
    for i in range(m):
        for j in range(p):
            for k in range(n):
                C[i][j] += A[i][k] * B[k][j]
    return C
```

```

def gaussian_elimination(A, b):
    n = len(b)
    Augmented = [A[i] + [b[i]] for i in range(n)]
    for i in range(n):
        max_row = i
        for j in range(i+1, n):
            if abs(Augmented[j][i]) > abs(Augmented[max_row][i]):
                max_row = j
        Augmented[i], Augmented[max_row] = Augmented[max_row], Augmented[i]
        factor = Augmented[i][i]
        for j in range(i, n+1):
            Augmented[i][j] /= factor
        for j in range(n):
            if i != j:
                factor = Augmented[j][i]
                for k in range(i, n+1):
                    Augmented[j][k] -= factor * Augmented[i][k]
    return [row[-1] for row in Augmented]

def is_quaternionic_representation(Q):
    # Placeholder function to check if Q is a quaternionic representation
    # This should be replaced with actual implementation
    return True

def xor_and_tree_width(f):
    # Placeholder function to compute XOR-AND tree width
    # This should be replaced with actual implementation
    return 10 # Example value

def run_trial(seed: int) -> dict:
    random.seed(seed)
    n = random.choice([10, 15, 20])
    f = [random.randint(0, 1) for _ in range(2**n)]

    # Compute quaternionic representation Q
    if not is_quaternionic_representation(f):
        return {
            "metric_name": "quaternionic_rank",
            "metric_value": -1,
            "instances_tested": 1,
            "conjecture_holds": False,
            "counterexample": "mapping_undefined"
        }

    # Compute XOR-AND tree width
    xor_and_width = xor_and_tree_width(f)

    # Check the conjecture
    quaternionic_rank = len(gaussian_elimination(f, [1]*n))
    conjecture_holds = quaternionic_rank <= xor_and_width

    return {
        "metric_name": "quaternionic_rank",
        "metric_value": quaternionic_rank,
        "instances_tested": 1,
        "conjecture_holds": conjecture_holds,
    }

```

```

        "counterexample": "" if conjecture_holds else f"rank={quaternionic_rank}, expected={xor_and}"
    }

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2**i + 17 for i in range(5, 8)]

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    if all(result["conjecture_holds"] for result in results):
        mean_value = sum(result["metric_value"] for result in results) / len(results)
        support_fraction = 1.0
        print(f"RESULT: SUPPORTED mean={mean_value} std=0 support_fraction={support_fraction}")
    else:
        first_failing_seed = next(seed for seed, result in zip(seeds, results) if not result["conjecture_holds"])
        counterexample = next(result["counterexample"] for result in results if not result["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample={counterexample} first_failing_seed={first_failing_seed}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fdd28fea.py", line 100, in <module>
    result = run_trial(seed)
              ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fdd28fea.py", line 83, in run_trial
    quaternionic_rank = len(gaussian_elimination(f, [1]*n))
                          ~~~~~^~~~~~
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_fdd28fea.py", line 37, in gaussian_elimination
    Augmented = [A[i] + [b[i]] for i in range(n)]
                  ~~~~~^~~~~~
TypeError: unsupported operand type(s) for +: 'int' and 'list'

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed during execution, which prevents us from verifying the conjecture’s support conditions. | next: Investigate and fix the error in the test code to allow for a proper verification of the conjecture.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	10940
2	preregistration	ollama_remote	glm4:latest	0	0	5414
3	novelty	ollama_remote	glm4:latest	0	0	4644
4	novelty	ollama_remote	glm4:latest	0	0	6353
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14167
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	11098
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	10830
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15905
9	judge	ollama_remote	glm4:latest	0	0	25748

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 105099 ms total latency. Provider mix: {'ollama_remote': 9}

(full prompt+response transcripts available in *research/audit/95d8ba0e68c2.jsonl*)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in *research/audit/95d8ba0e68c2.jsonl* for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: *research/pvsnp_sandbox/test_*.py* (per-cycle)

Replay tarball: *research/replay/95d8ba0e68c2.tar.gz* (if generated)